

ex5

December 8, 2025

```
[ ]: import numpy as np
import matplotlib.pyplot as plt
import math
```

this will be done through the recently learned gillepsie method, the steps are: 1 - calculate the propensities through $K_{\text{state}} \times (\text{number of occupants of the state})$ and the sum 2 - generate two random numbers r_1 and r_2 , and use them to select the next event and which time will it occur 3 - update the system, record the full state and then repeat it all again.

of course, we begin by selecting the starting conditions of the event

```
[ ]: t_i = 0.0  #starting time
t_f = 0.05 #finishing time

n_mono = 1000 #number of monomers
n_di = 0 #number of dimers

s_tot = 0

k_plus = 1.0  # A + A -> AA rate
k_minus = 1.0 # AA -> A + A rate

#record values
time = [t_i]
N_A = [n_mono]
N_AA = [n_di]
s_tot_vec = [s_tot]

t = t_i
while t < t_f:
    #calculate the propensities
    p_A = k_plus*(n_mono)*(n_mono-1)*0.5
    p_AA = k_minus*(n_di)
    p_0 = p_A+p_AA

    r1 = np.random.rand()
    r2 = np.random.rand()
```

```

tau = (1/p_0)*math.log(1/r1)

s_step = 0

if r2 < p_A/p_0:
    s_step = np.log(p_A/(n_di+1))
    n_mono -= 2
    n_di += 1
else:
    s_step = np.log(2*p_AA/((n_mono+2)*(n_mono+1)))
    n_di -= 1
    n_mono += 2

s_tot += s_step
t += tau

s_tot_vec.append(s_tot)
time.append(t)
N_A.append(n_mono)
N_AA.append(n_di)

lbd = np.sqrt(1+4*1000)
r_plus = (-1+lbd)/2.
r_minus = (-1-lbd)/2.
A = (1000-r_plus)/(1000-r_minus)

x = (r_plus - A*np.exp(-1*lbd*np.array(time))*r_minus)/(1 - A*np.exp(-lbd*np.
    ↪array(time))) #analytical solution to the occupation for monomers
y = (1000 - x)/2

plt.plot(time, x, linewidth=1, linestyle="--")
plt.plot(time, y, linewidth=1, linestyle="--")

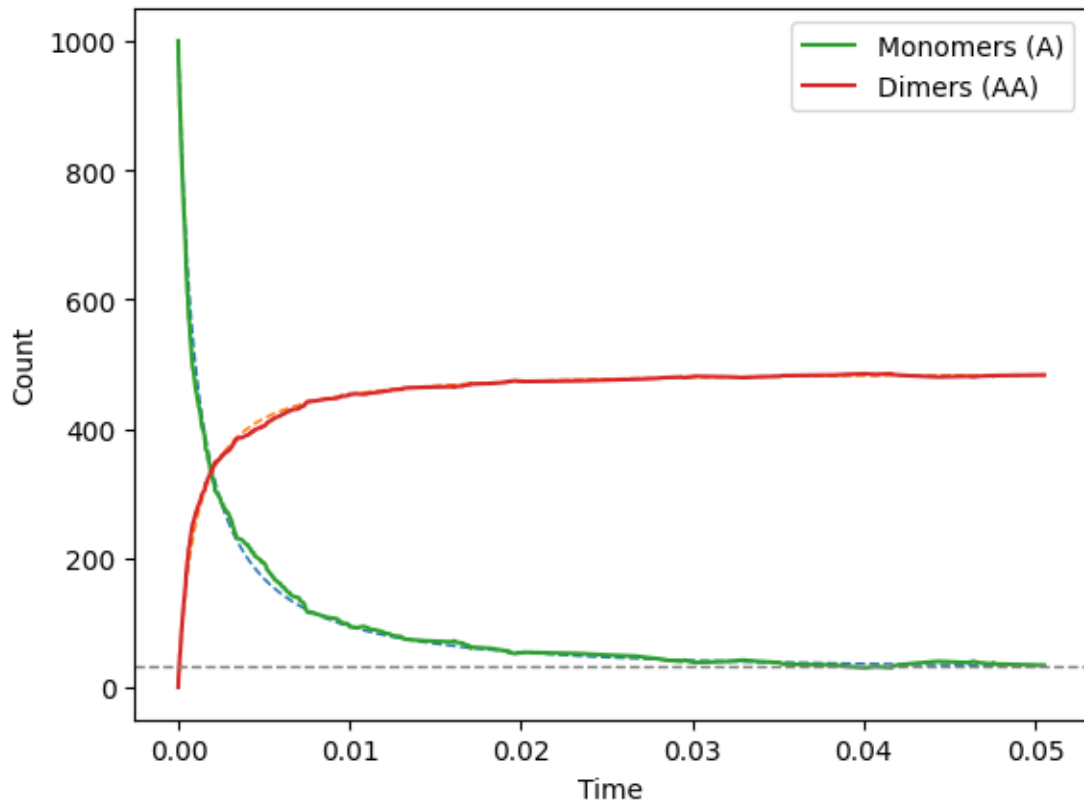
plt.plot(time, N_A, label='Monomers (A)')
plt.plot(time, N_AA, label='Dimers (AA)')
plt.axhline(y=r_plus, linestyle="--", color="gray", linewidth=1)

print(s_tot)

plt.xlabel('Time')
plt.ylabel('Count')
plt.legend()
plt.show()

```

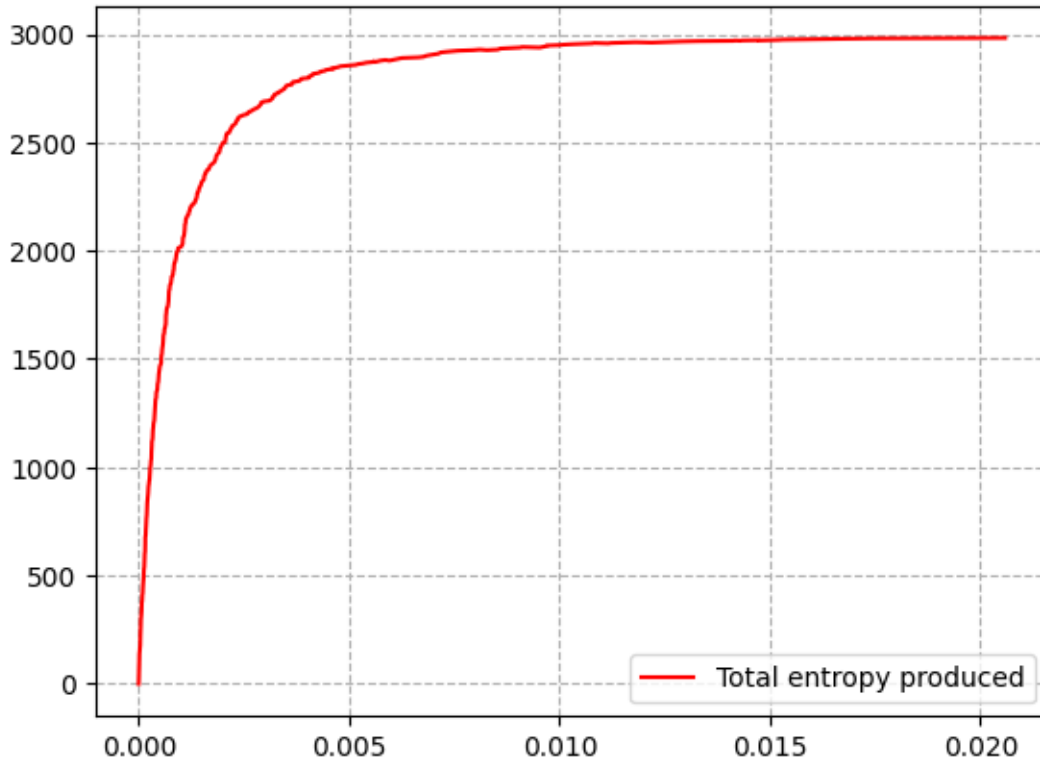
2982.80009962853



SIMULAÇÃO PARA ENTROPIA

```
[ ]: fig, ax = plt.subplots()
ax.plot(time, s_tot_vec, label="Total entropy produced", c="red")
ax.grid(True, linestyle='--')
ax.legend(fontsize=10)
```

```
[ ]: <matplotlib.legend.Legend at 0x7813799becb0>
```



utilizamos como $\dot{s}^{tot}$:

$$\dot{s}^{tot} = [w^+ - w^-] \cdot \log \frac{w^+}{w^-} \quad (1)$$

ver resto do exercicio para definicoes porque o latex do ipynb nao ta me deixando escrever aqui :/

```
[ ]: N=1000
lbd = np.sqrt(1+4*N)
r_plus = (-1+lbd)/2.
r_minus = (-1-lbd)/2.
A = (N-r_plus)/(N-r_minus)

def f(t):
    return (r_plus - A*np.exp(-1*lbd*t)*r_minus)/(1 - A*np.exp(-lbd*t))

def s_dot_mf(t):
    ff = f(t)
    w_plus = 0.5*ff*(ff-1)
    w_minus = 0.5*(N-ff)
    J = w_plus - w_minus
    return J * np.log(w_plus / w_minus)

time_array = np.linspace(0.001,0.02,100)
```

```

s_dot_tot_array = s_dot_mf(time_array)
s_dot_tot_array

import scipy.integrate as integrate
integrate.quad(s_dot_mf, 0.0, 0.1)

```

```
[ ]: (2976.398792182612, 4.0709614950174e-05)
```

```

[ ]: import numpy as np

kB = 1.0
N = 1000

def F(f):
    # careful when f is exactly N (N-f==0): use limit (N-f)*ln(N-f) -> 0
    term3 = 0.0 if np.isclose(N-f,0) else (N-f)*np.log(N-f)
    return f*np.log(f) + (f-1)*np.log(f-1) + term3 - (f + N)

def S_mf_of_t(f_t):
    f0 = N
    return 0.5*kB*(F(f0) - F(f_t))

# if you have your analytic f(t) function, compute:
t = 0.01
s = S_mf_of_t(f(t))

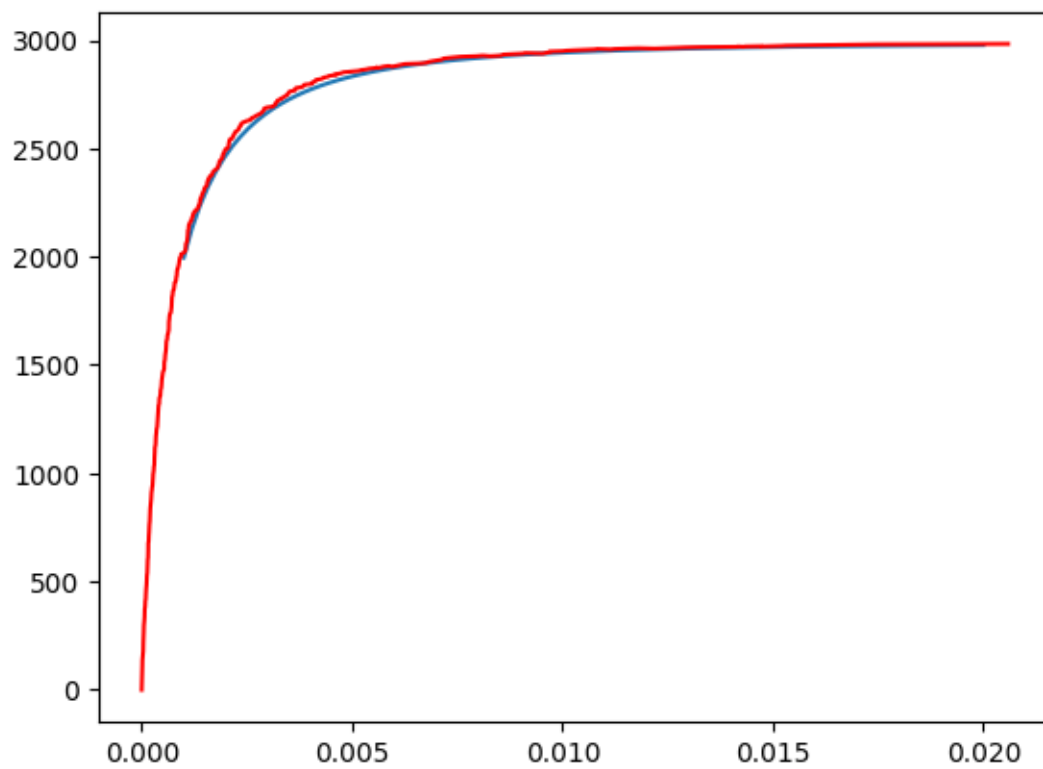
s_v = []

for t_i in time_array:
    s_aux = S_mf_of_t(f(t_i))
    s_v.append(s_aux)

plt.plot(time_array, s_v)
plt.plot(time, s_tot_vec, label="Total entropy produced", c="red")

```

```
[ ]: [<matplotlib.lines.Line2D at 0x78135f995f00>]
```



```
[ ]: f(0.05)
```

```
[ ]: np.float64(33.74303598717192)
```

```
[ ]:
```